

// JavaJub

Собес в Яндекс Путешествия Яндекс Вертикали Java Middle

Вопросы, задачи и подготовка
к live-coding и техническому интервью

Стек

Java · Kotlin · Spring Boot · PostgreSQL · YDB · gRPC · Logbroker · Kubernetes

[// содержание](#)

Содержание

01	Яндекс Путешествия и формат собеседа	3
02	Стек по вакансии	5
03	Java Core — что точно спросят	6
04	Коллекции	8
05	Многопоточность и JMM	9
06	Spring и Spring Boot	11
07	Hibernate и JPA	13
08	SQL и PostgreSQL	14
09	Алгоритмы — главный фильтр	16
10	System Design	19
11	Docker, Kubernetes, CI/CD	20
12	Практические задачи (live-coding)	21
13	План подготовки + чек-лист	24

01 Яндекс Путешествия и формат собеса

Яндекс Путешествия — часть бизнес-юнита Яндекс Вертикали (вместе с Авто.ру, Недвижимостью, Арендой). Это единственное крупное JVM-направление Яндекса наряду с Маркетом — можно прийти с обычным Java-бэкграундом, не переучиваясь на C++, Go или Scala.

Команда ~40+ бэкенд-разработчиков. Продукты: отели, авиа, ж/д, автобусы, B2B-командировки, экстранет для отельеров, биллинговая платформа. Нагрузка — десятки тысяч RPS, миллионы DAU.

► Как устроен процесс

Яндекс нанимает через общий поток «бэкенд в Путешествия». Уровень (Middle / Middle+ / Senior) определяется по итогам секций, а не указывается в вакансии. Секции оцениваются независимо — интервьюеры не обсуждают результаты до финальной калибровки.

Этап	Формат	Длительность
HR-скрининг	Zoom, мотивация, ЗП	30–60 мин
Техническое предварительное	Yandex.Code (без компиляции)	1 час
Advanced Code (с 2025)	IDE кандидата + автотесты + интернет	1 час
Алгоритмическая секция	Yandex.Code, 2–3 задачи	1 час
Java Core + многопоточность	Yandex.Code, теория + код	1–1.5 часа
System Design	Miro + Zoom	1 час (опц. для Middle)
Секция «про опыт» (STAR)	Разбор проектов	1 час
Финалы с командами	2–5 команд по 1 часу	2–5 часов

ФИШКА · Ключевая особенность

Можно провалить 1–2 секции и всё равно получить оффер — секции оцениваются независимо. В сомнительных случаях просто назначают дополнительную секцию. После отказа можно перезайти через 6–12 месяцев.

ВНИМАНИЕ · Про алгоритмы

Алгоритмическая секция — главный фильтр. Уровень сравним с Google. НО: Яндекс официально заявляет, что НЕ дают динамическое программирование, Дейкстру, КМР. Фокус — хэш-таблицы, два указателя, sliding window, BFS/DFS.

02 Стек по вакансии

Стек определён по вакансии «Разработчик на Java в Путешествия» (yandex.ru/jobs) и карточке на career.habr.com. Путешествия отличаются от остальных Вертикалей: Авто.ру пишет на Scala + ZIO, а Путешествия — на Java + Kotlin + Spring.

► Основной стек

- Java + Kotlin — основные языки (Go и C++ для отдельных компонентов)
- Spring / Spring Boot — основной фреймворк
- Hibernate + JOOQ — ORM и типизированный SQL
- PostgreSQL — OLTP-база
- YDB — распределённая база Яндекса (высокая нагрузка, отказоустойчивость)
- gRPC — внутренний RPC, HTTP/REST — наружу
- Logbroker — внутренняя шина данных (поверх YDB Topics, API-совместима с Kafka)
- YT / YTsaurus — аналитика, MapReduce, холодное хранилище
- Arcadia — монорепозитарий Яндекса
- Nanny / Yandex Deploy — деплой и оркестрация

► Что важно знать

- Требование: 5+ лет коммерческой разработки на Java или Kotlin
- Внутренние инструменты (YDB, Logbroker, Arcadia) осваиваются на месте — не нужно знать до собеседа
- Новичок проводит двухнедельный буткемп в каждой команде перед выбором
- Опционально: MultiTrack-буткемп — 8 недель работы в 3 командах с полной зарплатой

СОВЕТ · Для Java-разработчика

Путешествия — наиболее «дружественное» подразделение Яндекса для классического Java-разработчика: привычный стек Spring/Hibernate/PostgreSQL + Kotlin. Внутренние технологии (YDB, Logbroker, Arcadia) изучаются уже на месте.

03 Java Core — что точно спросят

По отзывам кандидатов (Medium, Habr 968968, Habr 854956): equals/hashCode + HashMap — абсолютные чемпионы. Далее — Stream API, generics, String Pool, ссылочные типы.

► Базовые вопросы

- JDK, JRE, JVM?
 - JVM — виртуальная машина (исполняет байт-код). JRE = JVM + стандартные библиотеки. JDK = JRE + инструменты (javac, jdb, jar). С Java 9 JRE отдельно не поставляется.
- Области памяти JVM.
 - Heap (Eden/Survivor/Old — объекты), Stack (фреймы методов), Metaspace (метаданные классов, раньше PermGen), PC Register, Native Method Stack, CodeCache (JIT-скомпилированный код).
- Контракт equals/hashCode.
 - Если `a.equals(b)`, то `hashCode` одинаков. Обратное необязательно. Рефлексивность, симметричность, транзитивность. `equals(null)` → `false`. Переопределяешь один — переопределяй оба. Нарушение ломает HashMap/HashSet.
- Что сломается, если `hashCode()` — константа?
 - Все объекты в одном bucket. Java 7: список $O(n)$. Java 8+: при 8 коллизиях И `capacity ≥ 64` — `red-black tree` $O(\log n)$. Деградация по сравнению с $O(1)$.
- String Pool и immutability.
 - Pool в heap хранит уникальные литералы. `new String("hi")` — новый объект вне пула. `intern()` добавляет в пул. Immutability: безопасность, потокобезопасность, кэширование `hashCode`, переиспользование в пуле.
- Generics: type erasure и PECS.
 - Джenericи стираются компилятором (type erasure) — в рантайме нет информации о типовом параметре. PECS: Producer Extends, Consumer Super. `List<? extends Number>` — читать, `List<? super Integer>` — писать.
- WeakReference / SoftReference / PhantomReference.
 - Weak — GC собирает при первой же сборке. Soft — GC собирает при нехватке памяти (для кэшей). Phantom — для постфинализации очистки ресурсов. WeakHashMap — ключи через WeakReference.
- Функциональные интерфейсы и Stream API.
 - Один абстрактный метод (SAM). Function, Predicate, Consumer, Supplier, Comparator. Стрим: промежуточные (ленивые) → терминальные (запускают конвейер). Обработка вертикальная, не горизонтальная. Стрим одноразовый.
- Optional — когда?
 - Для возвращаемых значений. НЕ для полей, параметров, коллекций. Антипаттерны: `get()` без `isPresent()`, Optional в полях, Optional<List>. `orElse` vs `orElseGet` (ленивый).
- final: класс, метод, поле.
 - Класс — нельзя наследовать. Метод — нельзя переопределить. Поле — нельзя переписать (но мутабельное содержимое можно менять). `effectively final` — для лямбд.
- Абстрактный класс vs интерфейс.
 - Абстрактный: состояние, конструкторы, один наследник. Интерфейс: контракт, множественная реализация. Java 8 — `default/static`. Java 9 — `private` методы.

► Задачи «Что выведет?»

Тест 1. Integer cache

```
Integer i1 = 127, i2 = 127;
System.out.println(i1 == i2); // ?

Integer i3 = 128, i4 = 128;
System.out.println(i3 == i4); // ?
```

Ответ: true, false. IntegerCache: -128..127. Для 127 — один объект, ссылки совпадают. Для 128 — два разных. Мораль: для объектов — equals(), никогда ==.

Тест 2. Stream — ленивость

```
List.of(1,2,3,4,5).stream()
    .map(x -> { System.out.print(x+" "); return x; })
    .filter(x -> x > 2)
    .map(x -> { System.out.print(x+" "); return x; })
    .toList();
```

Ответ: 1 2 3 3 4 4 5 5. Стрим обрабатывает вертикально: каждый элемент проходит всю цепочку. 1 и 2 не проходят filter → печатаются раз. 3,4,5 проходят → дважды.

Тест 3. Передача по значению

```
Integer i = Integer.valueOf(1);
inc(i);
System.out.println(i); // ?

static void inc(Integer i) { i++; }
```

Ответ: 1. Java передаёт ссылки по значению. i++ создаёт новый Integer(2) и присваивает локальной переменной. Оригинал не меняется.

СОВЕТ · Лайфхак

На вопрос про equals/hashCode приведи практический пример: «положить объект в HashSet, изменить поле в hashCode — объект потеряется, contains() вернёт false». Это показывает понимание, а не заученность.

04 Коллекции

HashMap — чемпион. По опыту кандидатов в Яндекс: «обсасывают мапу со всех сторон — устройство, коллизии, treeify, resize, ключи-массивы, что будет если положить и не найти».

- Как устроен HashMap?
 - Массив `Node<K,V>[]`. Размер — степень двойки (default 16). Индекс: $(n-1) \& \text{hash}(\text{key})$, `hash` — XOR верхних 16 бит с нижними. Коллизии — связный список. Java 8+: при `TREEIFY_THRESHOLD=8` и `capacity ≥ 64` → *red-black tree*. При 6 → обратно (*UNTREEIFY*).
- load factor и resize.
 - Порог 0.75. При `size ≥ capacity * loadFactor` — *resize*: массив вдвое + перехеширование ВСЕХ элементов. Совет: `initialCapacity = expectedSize / 0.75 + 1`.
- HashMap vs ConcurrentHashMap.
 - *HashMap*: не потокобезопасен, допускает `null`-ключ. *ConcurrentHashMap*: Java 7 — *Segment-блокировки*, Java 8+ — *CAS + synchronized* на головах бакетов. `null` запрещён. `computeIfAbsent` — атомарная операция.
- ArrayList vs LinkedList.
 - *ArrayList*: $O(1)$ доступ, $O(n)$ вставка в середину, дружелюбен к CPU-кэшу. *LinkedList*: $O(1)$ вставка/удаление в начало, $O(n)$ доступ. На практике *ArrayList* почти всегда лучше.
- TreeMap vs LinkedHashMap.
 - *TreeMap*: *red-black tree*, $O(\log n)$, ключи отсортированы. *LinkedHashMap*: *HashMap* + двусвязный список, порядок вставки. `accessOrder=true` → LRU-кэш.
- Fail-fast итератор.
 - *ConcurrentModificationException* при изменении коллекции не через итератор. `modCount`-счётчик. Не гарантирован в многопоточке. Альтернатива: *CopyOnWriteArrayList* (*fail-safe*).
- Можно ли `byte[]` как ключ HashMap?
 - Технически можно, но `hashCode` у массива — по адресу (`identityHashCode`), а `equals` — по ссылке. Два одинаковых по содержимому массива дадут разные хэши. Нужно обернуть в *ByteBuffer* или свой класс с переопределёнными `equals/hashCode`.

05 Многопоточность и JMM

В Яндексе многопоточность спрашивают глубоко — `volatile` vs `synchronized` давали на каждом собеседовании. JMM, happens-before, CAS, пулы потоков — обязательно.

- `synchronized`.
 - Захват монитора. *Instance-метод* → *this*. *Static* → *Class*. Блок → указанный объект. *Reentrant* (счётчик). Гарантирует: *mutual exclusion* + *happens-before* (видимость).
- `volatile`.
 - Видимость (запрет кэширования в регистрах/L1) + запрет *reordering*. НЕ гарантирует атомарность `i++` (*read-increment-write*). Для атомарных — `AtomicInteger/LongAdder`.
- `happens-before`.
 - `unlock` → `lock` того же монитора. `volatile write` → `read`. `Thread.start()` → первая инструкция потока. Последняя инструкция → `join()`. `final`-поля видны после конструктора.
- `ConcurrentHashMap`: Java 7 vs Java 8.
 - Java 7: массив `Segment` (`ReentrantLock`), каждый сегмент — свой `HashMap`. Java 8: убрали `Segment`, `CAS` + `synchronized` на головах бакетов (лучше параллелизм). *Treeify* при 8 коллизиях.
- `ThreadPoolExecutor` — параметры.
 - `corePoolSize`, `maxPoolSize`, `keepAliveTime`, `workQueue` (`LinkedBlockingQueue/ArrayBlockingQueue/SynchronousQueue`), `threadFactory`, `rejectedExecutionHandler` (`AbortPolicy/CallerRunsPolicy/DiscardPolicy/DiscardOldestPolicy`).
- `CountDownLatch` vs `CyclicBarrier` vs `Semaphore`.
 - `CountDownLatch`: одноразовый счётчик, потоки ждут обнуления. `CyclicBarrier`: переиспользуемый, все потоки ждут друг друга. `Semaphore`: ограничение числа одновременных потоков (пул ресурсов).
- `Deadlock` — как возникает?
 - Два+ потока ждут друг друга. Условия Коффмана. Избежать: упорядочить захват мониторов, `tryLock` с таймаутом. Обнаружение: `jstack`, `VisualVM`, `Thread.getState()`.
- Реализуй ограниченную `BlockingQueue`.
 - `synchronized` + `wait/notify`. `while(!condition) wait()` — не `if!` (*spurious wakeups*). `put()` ждёт если полная, `take()` ждёт если пустая. `notifyAll()` после каждой операции.

ЛОВУШКА · `volatile counter++`

`volatile long counter; counter++` — НЕБЕЖНО. Это три операции (*read-increment-write*), `volatile` не обеспечивает атомарность. Три корректных варианта: `synchronized`, `AtomicLong.incrementAndGet()`, `LongAdder.increment()`.

06 Spring и Spring Boot

Spring — основной фреймворк Путешествий. Спрашивают глубоко: прокси, жизненный цикл бинов, автоконфигурация, AOP.

- IoC и DI.
 - IoC — контейнер управляет жизненным циклом. DI — зависимости внедряются извне. Три способа: конструктор (лучший), сеттер, поле (`@Autowired`). Constructor injection: поля `final`, явные зависимости, легко тестировать.
- Жизненный цикл бина.
 - `BeanDefinition` → инстанцирование → DI → Aware-интерфейсы → `BPP.postProcessBefore` → `@PostConstruct` → `InitializingBean` → `init-method` → `BPP.postProcessAfter` → ГОТОВ → `@PreDestroy` → `DisposableBean` → `destroy-method`. BPP — точка создания прокси.
- Scope бинов.
 - `singleton` (default), `prototype`, `request`, `session`, `application`, `websocket`. Инжект `prototype` в `singleton`: через `Provider<T>`, `ObjectFactory<T>` или `@Lookup`. Иначе `prototype` создастся один раз.
- `@Transactional` — прокси.
 - Spring создаёт прокси (JDK/CGLIB). Прокси открывает транзакцию, вызывает метод, `commit/rollback`. `self-call` минует прокси → `@Transactional/@Cacheable/@Async` не работают. Решение: вынести в другой бин.
- Автоконфигурация Spring Boot.
 - `@EnableAutoConfiguration` + `@Conditional`. Если `DataSource` в `classpath` — автоматически настроит JPA. Список: `META-INF/spring/...AutoConfiguration.imports` (Spring Boot 3+, раньше `spring.factories`).
- `@SpringBootApplication`.
 - `@Configuration` + `@EnableAutoConfiguration` + `@ComponentScan`.
- AOP: JDK proxy vs CGLIB.
 - JDK dynamic proxy — если бин реализует интерфейс (через `Proxy.newProxyInstance`). CGLIB — наследование от класса (`final`-классы нельзя проксировать). Spring Boot 3 по умолчанию использует CGLIB.
- Обработка исключений.
 - `@RestControllerAdvice` + `@ExceptionHandler`. `ErrorDto`: `code`, `message`, `timestamp`. HTTP-статусы: 400 (валидация), 404, 409 (конфликт), 500.

07 Hibernate и JPA

- Состояния сущности.
→ *Transient* → *Persistent (persist)* → *Detached (close/detach)* → *Removed (remove)*. *Persistent: dirty checking* — изменения автоматически синхронизируются.
- N+1 проблема.
→ *1 findAll() + N доп. запросов на lazy-коллекции*. Решения: *JOIN FETCH*, *@EntityGraph*, *@BatchSize(100)*, DTO-проекция. *EAGER* — неправильный ответ.
- *LazyInitializationException*.
→ Обращение к *lazy*-полю после закрытия сессии. Решения: *JOIN FETCH*, *@EntityGraph*, DTO. *OpenSessionInView* — анти-паттерн.
- Оптимистичная блокировка.
→ *@Version (int/long)*. *UPDATE ... WHERE version = ?*. Не совпала → *OptimisticLockException*. Для *low-contention* (веб-приложения). Пессимистичная: *SELECT FOR UPDATE*.
- JOOQ vs Hibernate.
→ *JOOQ*: типизированный SQL, *compile-time* проверка запросов, нет *dirty checking*. *Hibernate*: ORM, *lazy loading*, кэширование. В Путешествиях используют оба — *JOOQ* для сложных запросов, *Hibernate* для CRUD.
- Кэши Hibernate.
→ *First-level*: привязан к *Session*, автоматический. *Second-level*: общий (*EhCache*, *Caffeine*), опциональный. *Query cache*: кэширует JPQL-результаты.

ЛОВУШКА · EAGER = решение N+1?

Нет. EAGER грузит коллекцию ВСЕГДА, даже когда она не нужна — медленнее и съедает память. Правильно: LAZY по умолчанию + FETCH/EntityGraph там, где нужны данные.

08 SQL и PostgreSQL

SQL спрашивают отдельной задачей — пишешь запросы вживую. Оконные функции, EXPLAIN ANALYZE, уровни изоляции — обязательно для Яндекса.

- ACID.
 - *Atomicity (целиком/никак), Consistency (валидное состояние), Isolation (изоляция транзакций), Durability (после COMMIT данные сохраняются — WAL в PostgreSQL).*
- Уровни изоляции.
 - *READ_UNCOMMITTED, READ_COMMITTED (default PG), REPEATABLE_READ, SERIALIZABLE. Аномалии: dirty read, non-repeatable read, phantom read, serialization anomaly.*
- MVCC в PostgreSQL.
 - *Каждая строка: xmin (создавшая транзакция), xmax (удалившая). Читатели не блокируют писателей. Старые версии чистит VACUUM. Проблема: table bloat.*
- Типы индексов PostgreSQL.
 - *B-tree (default), Hash (=), GIN (JSONB, full-text), GiST (геометрия), BRIN (большие таблицы с порядком). Покрывающий индекс (INCLUDE) — Index Only Scan.*
- Оконные функции.
 - *ROW_NUMBER(), RANK(), DENSE_RANK(), LAG/LEAD, SUM/AVG OVER (PARTITION BY ... ORDER BY ...). Агрегация без GROUP BY — строки не сворачиваются.*
- EXPLAIN ANALYZE.
 - *Seq Scan (плохо на большой таблице), Index Scan (хорошо), Bitmap Heap Scan. Rows Removed by Filter — индекс не помогает. estimated ≠ actual → ANALYZE.*

► SQL-задачи из собеседов Яндекса

Вторая по величине зарплата (3 способа)

```
-- Способ 1: вложенный MAX
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);

-- Способ 2: DISTINCT + OFFSET
SELECT DISTINCT salary FROM employees
ORDER BY salary DESC LIMIT 1 OFFSET 1;

-- Способ 3: DENSE_RANK
SELECT salary FROM (
  SELECT salary, DENSE_RANK() OVER (ORDER BY salary DESC) AS rk
  FROM employees) t WHERE rk = 2;
```

Топ-5 артикулов в каждом регионе

```
SELECT * FROM (
  SELECT region, article, sales,
    ROW_NUMBER() OVER (PARTITION BY region ORDER BY sales DESC) AS rn
  FROM sales_data
) t WHERE rn <= 5;
```

09 Алгоритмы — главный фильтр

Алгоритмическая секция — самая обсуждаемая часть. 2–3 задачи за 1 час, решения в 10–30 строк. Код пишется в Yandex.Code — без компиляции, без автокомплита. Ключевое: умение «прогнать» код в голове.

► Что НЕ спрашивают (официально)

- Динамическое программирование
- Алгоритм Дейкстры
- KMP (поиск подстроки)
- Собственные реализации куч
- Сложные графовые алгоритмы

► Что спрашивают (по отзывам)

- Хэш-таблицы: Two Sum, подсчёт частот, анаграммы, группировка
- Два указателя: сжатие интервалов, удаление дубликатов, палиндромы
- Sliding window: детектор роботов (>5000 событий/час через Deque)
- BFS/DFS: Number of Islands, обход графа/дерева
- Сортировка + бинарный поиск: Search in Rotated Sorted Array
- Стек/очередь: скобочные последовательности, монотонный стек
- Строки: Run-Length encoding (aaaffcccd → 3a2f4cd)
- Слияние: Merge k sorted arrays через PriorityQueue

► Реальные задачи из собеседований Яндекса

Задача 1. Сжатие в диапазоны

Дан массив [1,4,5,2,3,9,8,11,0]. Вернуть строку "0-5,8-9,11".

```
public String compressRanges(int[] arr) {
    Arrays.sort(arr);
    StringBuilder sb = new StringBuilder();
    int i = 0;
    while (i < arr.length) {
        int start = arr[i];
        while (i + 1 < arr.length && arr[i+1] == arr[i] + 1) i++;
        if (arr[i] == start) sb.append(start);
        else sb.append(start).append('-').append(arr[i]);
        sb.append(',');
        i++;
    }
    sb.setLength(sb.length() - 1);
    return sb.toString();
}
```

$O(n \log n)$ из-за сортировки. Два указателя для расширения диапазона.

Задача 2. Run-Length Encoding

```
public String rle(String s) {
    StringBuilder sb = new StringBuilder();
```

```

int i = 0;
while (i < s.length()) {
    char c = s.charAt(i);
    int count = 0;
    while (i < s.length() && s.charAt(i) == c) {
        count++; i++;
    }
    if (count > 1) sb.append(count);
    sb.append(c);
}
return sb.toString();
}

```

aaaffcccd → 3a2f4cd. $O(n)$ времени и памяти.

Задача 3. Merge Intervals

```

public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]);
    List<int[]> result = new ArrayList<>();
    int[] current = intervals[0];
    for (int i = 1; i < intervals.length; i++) {
        if (intervals[i][0] <= current[1]) {
            current[1] = Math.max(current[1], intervals[i][1]);
        } else {
            result.add(current);
            current = intervals[i];
        }
    }
    result.add(current);
    return result.toArray(new int[0][]);
}

```

$O(n \log n)$. Сортировка по началу + жадное слияние.

Задача 4. Детектор роботов (sliding window)

Дан поток событий (userId, timestamp). Найти пользователей с >5000 событиями за любой час.

```

Map<Long, Deque<Long>> windows = new HashMap<>();

void processEvent(long userId, long timestamp) {
    windows.computeIfAbsent(userId, k -> new ArrayDeque<>());
    Deque<Long> deque = windows.get(userId);
    deque.addLast(timestamp);
    while (deque.peekFirst() < timestamp - 3600) {
        deque.pollFirst();
    }
    if (deque.size() > 5000) {
        markAsBot(userId);
    }
}

```

ФИШКА · Тренировочный контекст

Официальный контекст Яндекса — contest.yandex.ru/contest/8458. 6 задач: камни, последовательные единицы, дубликаты, скобки, анаграммы, слияние массивов. Прорешай его перед собесом.

10 System Design

Для Middle секция необязательна, но сильный результат поднимет грейд (Middle → Middle+). Для Senior — ключевая. Кандидат ведёт обсуждение, интервьюер задаёт наводящие вопросы.

► Что оценивают

- Уточнение требований: функциональные + нефункциональные (RPS, SLA, объёмы)
- Back-of-envelope расчёты (сколько запросов, сколько данных, сколько серверов)
- High-level архитектура (компоненты, потоки данных, API)
- Обоснование выбора технологий (SQL vs NoSQL, sync vs async)
- Масштабирование и шардирование
- Отказоустойчивость и консистентность

► Реальные задачи из Яндекса

- Сервис коротких ссылок.
→ *base62 с $62^7 \approx 3.5$ трлн. Redis-кэш популярных. Расчёт: чтения/записи 100:1. Шардирование по хэшу ключа.*
- Распределённый rate limiter.
→ *Token Bucket vs Sliding Window Log. Redis + Lua-скрипт для атомарности. Обсудить: точность vs производительность.*
- Яндекс Go / такси-сервис.
→ *Геоиндекс (Geohash/H3), matching-сервис, партиционирование по городам, eventual consistency геопозиций.*
- Client-side Load Balancer.
→ *Round-robin → retry → circuit breaker. Обсудить: health checks, backoff, fallback. Реальная задача Middle в Путешествиях.*
- Сервис бронирования отелей.
→ *Поиск с динамическими ценами. Outbox-паттерн для финансовых операций. Идемпотентность платежей. Партнёрские интеграции через API Gateway.*

11 Docker, Kubernetes, CI/CD

Яндекс использует свои инструменты деплоя (Nanny / Yandex Deploy), но принципы контейнеризации и CI/CD спрашивают. Docker и K8s — must-have знание.

- Образ vs контейнер.
→ Образ — неизменяемый шаблон из слоёв. Контейнер — запущенный экземпляр с writable-слоем.
- Multi-stage build.
→ Сборка (Maven/Gradle) → финальный образ только JAR. ~200 MB вместо ~800 MB. Безопаснее, быстрее.
- Kubernetes: Pod, Deployment, Service.
→ Pod — 1+ контейнер. Deployment — реплики, rolling update. Service — стабильный endpoint (ClusterIP/NodePort/LoadBalancer). Ingress — L7.
- Liveness vs Readiness probe.
→ Liveness: жив ли? Нет → перезапуск. Readiness: готов к трафику? Нет → убирается из балансировки. Spring Boot Actuator: /health/liveness, /health/readiness.
- CI/CD pipeline.
→ checkout → build → test → docker build → push registry → deploy staging → (approval) → prod. В Яндексе: Arcadia CI + Nanny для деплоя.

12 Практические задачи (live-coding)

Помимо алгоритмов, на собеседовании в Яндекс могут попросить написать REST-контроллер, unit-тест, или решить задачу на Stream API. С 2025 года — секция Advanced Code: IDE + автотесты + интернет.

► Задача 1. Группировка Stream API

```
// Отдел → список сотрудников
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

// Отдел → средняя зарплата
Map<String, Double> avg = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)));
```

► Задача 2. Потокобезопасный LRU-кэш

```
public class LRUCache<K, V> {
    private final int capacity;
    private final Map<K, V> map;

    public LRUCache(int capacity) {
        this.capacity = capacity;
        this.map = new LinkedHashMap<>(capacity, 0.75f, true) {
            @Override
            protected boolean removeEldestEntry(Map.Entry<K, V> e) {
                return size() > capacity;
            }
        };
    }

    public synchronized V get(K key) { return map.get(key); }
    public synchronized void put(K key, V value) { map.put(key, value); }
}
```

LinkedHashMap с accessOrder=true + removeEldestEntry. synchronized для потокобезопасности. Для высокой нагрузки — Caffeine.

► Задача 3. REST-контроллер

```
@RestController
@RequestMapping("/api/bookings")
public class BookingController {

    private final BookingService bookingService;

    public BookingController(BookingService bookingService) {
        this.bookingService = bookingService;
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
```



```

    public BookingDto create(@RequestBody @Valid CreateBookingRequest req) {
        return bookingService.create(req);
    }

    @GetMapping("/{id}")
    public BookingDto getById(@PathVariable Long id) {
        return bookingService.findById(id);
    }
}

```

► Задача 4. Unit-тест Mockito + AssertJ

```

@ExtendWith(MockitoExtension.class)
class BookingServiceTest {
    @Mock BookingRepository repo;
    @Mock PaymentGateway gateway;
    @InjectMocks BookingService service;

    @Test
    void shouldCreateBookingAndCharge() {
        // given
        var req = new CreateBookingRequest("hotel-1", LocalDate.now());
        var saved = new Booking(1L, "hotel-1", Status.CONFIRMED);
        when(repo.save(any())).thenReturn(saved);
        // when
        var result = service.create(req);
        // then
        assertThat(result.getStatus()).isEqualTo(Status.CONFIRMED);
        verify(gateway).charge(any());
    }
}

```

► Задача 5. Code Review — найди проблемы

```

public class UserCache {
    private static Map<Long, User> cache = new HashMap<>();

    public static User getUser(Long id) {
        if (cache.containsKey(id)) {
            return cache.get(id);
        }
        User user = loadFromDb(id);
        cache.put(id, user);
        return user;
    }
}

```

Проблемы: 1) HashMap не потокобезопасен → ConcurrentHashMap. 2) containsKey+get → computeIfAbsent. 3) Кэш бесконечный → memory leak. 4) static → тяжело тестировать. 5) null от loadFromDb.

13 План подготовки + чек-лист

► За 3–4 недели

- Прорешать contest.yandex.ru/contest/8458 — официальный констест Яндекса
- 30–50 задач LeetCode Easy+Medium: hash-table, two-pointers, sliding-window, BFS/DFS
- Бесплатный купс start.practicum.yandex/algorithms-interview
- Повторить Java Concurrency in Practice (Гётц) — volatile, synchronized, CAS, пулы
- Поднять проект Spring Boot 3 + PostgreSQL + gRPC + тесты

► За неделю

- 2–3 мок-интервью: pramp.com, interviewing.io или друзья
- Прорешать все вопросы из гайда ВСЛУХ — мысли в голове ≠ слова
- Подготовить 2–3 проекта по STAR: проблема → что сделал → результат
- Потренироваться писать код БЕЗ IDE — именно так будет на секции
- Решить 5+ задач на бумаге/в блокноте и «прогнать» в голове

► В день собеседа

- Камера, микрофон, интернет — проверить за 30 минут
- Yandex.Code не поддерживает автокомплит — привыкни заранее
- Рассуждать вслух — молчание хуже «дай подумать»
- Сначала — план решения, потом — код. Не бросайся писать сразу
- Обязательно «прогони» код на примере — баги в голове = отказ
- 2–3 вопроса в конце: про команду, проект, стек, буткемп

ВНИМАНИЕ • Главная причина отказов

Не знание теории, а невнятное изложение решения и баги, которые кандидат не замечает при прогоне кода в голове. Тренируйтесь писать и проверять код без IDE!

► Финальный чек-лист

Блок	Готов, если можешь...
Java Core	equals/hashCode на примере + Integer cache + generics PECS
Коллекции	HashMap в Java 8+: treeify, resize, mutable-ключ, byte[] как ключ
Многопоточность	BlockingQueue через wait/notify + volatile vs synchronized + CAS
JMM	happens-before на 5+ примерах, почему volatile counter++ некорректно

Spring	Жизненный цикл бина наизусть + self-call + JDK vs CGLIB прокси
Hibernate	N+1 → JOIN FETCH / @EntityGraph / DTO. JOOQ vs Hibernate — когда что
SQL	Оконные функции + EXPLAIN ANALYZE + вторая зарплата 3 способами
Алгоритмы	3 задачи за 1 час в текстовом редакторе БЕЗ компиляции
System Design	Short URL или Rate Limiter с расчётами RPS и выбором технологий
Live-coding	LRU-кэш + REST-контроллер + code review за 15 минут каждое

Удачи на собеседе!

// git push origin offer

JavaJub